

Laboratorio 3 · Aprender a interpolar

Un agente que descubre dónde poner los nodos, y tu primer teorema para la biblioteca del curso

Métodos numéricos · Matemáticas · 90 minutos · en parejas

El jueves vimos Taylor y Lagrange: aproximar es decidir dónde se pone la información. Taylor la pone toda en un punto; Lagrange la reparte en nodos. Hoy vas a sacar los basales a mano, graficarlos, usarlos con el PIB de tres países, ver a un agente descubrir **dónde** poner los nodos sin que nadie le hable de Chebyshev, y escribir tu primer teorema como grafo para la biblioteca de demostraciones del curso.

Todas las órdenes se escriben dentro de `07_interpolacion` y están escritas para Windows.

Si usas Mac o Linux

Windows

`.venv\Scripts\python`

`..\docs\curso\lab03\experimentos.py`

Mac / Linux

`.venv/bin/python`

`../docs/curso/lab03/experimentos.py`

Qué vas a hacer

1. Construir a mano los polinomios basales y descubrir qué mide la suma de sus valores absolutos.
2. Ajustar un programa corto para graficarlos y usarlos con datos reales: el PIB de Vietnam, Colombia y China.
3. Ver a un agente aprender que los nodos de Chebyshev ganan, y romperlo para entender por qué.
4. Escribir el grafo de un teorema y ponerlo a prueba con el mismo verificador del agente.

1 Instalación

0-10 min

```
cd 07_interpolacion
python -m venv .venv
.venv\Scripts\python -m pip install -e ".[dev]" matplotlib
.venv\Scripts\python -m interprl --no-tui --seed 1
```

El ZIP tiene que ser de hoy: si lo descargaste antes, no trae `docs\curso\lab03`. Cada proyecto tiene su propio `.venv`. La última orden tarda unos 20 segundos (interpola en una malla fina) y termina imprimiendo un informe. No está colgada: mientras corre, empieza la parte 2.

2 Los basales a mano

10-25 min, en papel, sin IA

Para nodos distintos $t_0, \dots, t_n$, el basal L_k vale 1 en t_k y 0 en los demás nodos:

$$L_k(t) = \prod_{j \neq k} \frac{t - t_j}{t_k - t_j}, \quad P(t) = \sum_{k=0}^n f_k L_k(t).$$

- (a) Con los nodos $-1, 0, 1$, escribe L_0, L_1 y L_2 como producto y después expandidos. Verifica en los tres nodos que valen 1 en el suyo y 0 en los otros.

- (b) Lo mismo con los nodos 0, 1, 3. Fíjate en el signo del denominador de L_1 .
- (c) Llena la tabla. **Antes de la última fila**, escribe al margen si esperas valores entre 0 y 1.

nodos	t	L_0	L_1	L_2	suma	suma de $ L_k $
-1, 0, 1	-1/2					
-1, 0, 1	1/2					
0, 1, 3	2					
-1, 0, 1	2					

- (d) La columna «suma» te dio siempre lo mismo. Demuestra que, para cualesquiera $n+1$ nodos distintos, $L_0(t) + \dots + L_n(t) = 1$ para todo t . Escríbela en pasos: cada paso es **una afirmación y la razón** que la justifica (una definición, un teorema o una cuenta). Pista: ¿qué polinomio de grado $\leq n$ vale 1 en todos los nodos? ¿Qué dice la unicidad?

Revisa tu tabla con el programa (da fracciones exactas):

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py basales
```

La columna «suma de $|L_k|$ » tiene nombre: es la **función de Lebesgue** $\lambda(t)$, y dice cuánto puede amplificar el polinomio un error en los datos. Su máximo en el intervalo es la **constante de Lebesgue** Λ . Para -1, 0, 1 es exactamente uno de los números que acabas de calcular a mano.

3 Graficar y usar

25-40 min

Abre `..\docs\curso\lab03\graficar.py` con el Bloc de notas. Solo se cambia el bloque **AJUSTA AQUÍ**. Córrelo así:

```
.venv\Scripts\python ..\docs\curso\lab03\graficar.py
```

Guarda las figuras en `runs\lab03`. Ábrelas desde el explorador de archivos.

1. Córrelo tal como está. Compara `basales.png` con lo que dibujarías tú.
2. Agrega 2 a **PUNTOS**. La línea vertical no aparece. Averigua por qué y cambia **RANGO**.
3. Cambia **NODOS** a `[0, 1, 3]` y **PUNTOS** a `[2]`.

La segunda figura usa los mismos basales con el PIB (miles de millones de USD corrientes, Banco Mundial). Cambiamos de variable para que los nodos queden en números pequeños: $t = (\text{año} - \text{centro})/\text{escala}$.

ejemplo	nodos (año: PIB)	cambio de variable	estimar
Vietnam	2012: 195.6 · 2014: 233.5 · 2016: 257.1	$t = (\text{año} - 2014)/2$	2013 y 2015
Colombia	2018: 334.2 · 2019: 323.0 · 2021: 318.5	$t = \text{año} - 2018$	2020
China	2017: 12 310.5 · 2019: 14 280.0 · 2021: 17 820.5	$t = (\text{año} - 2019)/2$	2022 y 2023

En cada ejemplo, **primero calcula P a mano** con tu tabla de (c), después corre el programa. Vietnam ya está configurado; en Colombia y China llenas tú los campos que dicen **None**.

- (e) Vietnam: tus dos estimaciones y su error. ¿Por qué funcionó tan bien?

- (f) Colombia: ¿qué cree el polinomio que pasó en 2020? ¿Qué pasó? El error de interpolación depende de $f'''(\xi)$: ¿por qué esa fórmula no te salva aquí?
- (g) China: compara el error de 2023 con el de 2022. Si el dato de 2021 tuviera un error del 1 %, ¿cuánto se movería la estimación de 2023? Usa tu última fila de (c).

4 Mira cómo aprende

40-55 min

```
.venv\Scripts\python -m interpr1 --seed 1
```

El juego: hay que aproximar una f en $[-1, 1]$ con error menor que 10^{-5} usando el menor número de nodos. El agente decide dos cosas:

- **qué familia de nodos**, una vez por partida: $\gamma = 0$ son equiespaciados, $\gamma = 1$ son los de Chebyshev, y en medio hay mezclas. El agente no sabe qué es Chebyshev;
- **añadir otro nodo o entregar**, mirando solo cuánto cambió el polinomio respecto al anterior, $|p_n - p_{n-1}|$, si ese cambio va bajando, y si lleva pocos o muchos nodos. Nunca ve f ni el error real.

Puntos: -1 por cada nodo más lo que haya bajado el error; $+10$ si entrega con el error pedido; -10 si entrega antes de tiempo. Anota la generación de cada hito y repite con `--seed 2`.

hito	gen. (seed 1)	gen. (seed 2)
primera aproximación correcta		
descubrió el criterio de parada		
su regla coincide con la teoría		
prefiere $\gamma = 1$ (Chebyshev) de forma estable		
primera demostración completa		
primera demostración mínima		

Abre `runs\<fecha>\informe.md` con el Bloc de notas.

- (h) En la tabla de $Q(\gamma)$, ¿qué significa que $Q(\gamma = 1)$ sea mucho mayor que $Q(\gamma = 0)$? Mira la tabla de Λ del informe y relaciónala con tu respuesta de (g).
- (i) El agente decide cuándo parar viendo $|p_n - p_{n-1}|$, no el error real. ¿Por qué es un buen estimador? ¿Puede engañarlo? Busca en el informe cuántas veces entregó antes de tiempo.

5 Rómpelo

55-70 min

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py familias
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py solo_equi
```

- (j) `familias` no usa agente: para cada tipo de función dice cuántas veces cada familia logra el error pedido. ¿Con qué funciones fallan los equiespaciados? En `solo_equi`, ¿cuánto cae el éxito? ¿Por qué el agente prefiere entregar mal que seguir añadiendo nodos?

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py sin_runge
```

- (k) Ahora el agente solo ve funciones fáciles (exponenciales y senos). ¿Sigue prefiriendo Chebyshev? Lo que aprendió en la parte 4, ¿era una propiedad de los nodos o de los problemas que le mostramos?

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py recompensa
```

- (l) En el caso «entregar mal no cuesta», ¿qué aprende? ¿Por qué es racional desde los puntos? (Pista: compara una partida que entrega en el primer paso con una que trabaja 18 nodos y gana +10.)

Si te sobra tiempo, inventa tu recompensa:

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py recompensa --entregar-mal 0
--nodo -1 --entregar-bien 10
```

6 La demostración como grafo

70-85 min

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py dibujar
```

Es el teorema del error: 13 pasos y 8 distractores, en `interp\proof_kb.py`.

- (m) El distractor `D_TAYLOR` dice $f(x) - p_n(x) = f^{(n+1)}(\xi)(x - x_0)^{n+1}/(n+1)!$. Eso es el resto de Taylor. ¿Qué tendría que pasarles a los nodos para que fuera cierto?

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py grafo
```

Quitamos `FIX` (« x no es un nodo, luego $w(x) \neq 0$ ») de las dependencias de `AUX`. El verificador acepta una demostración de 12 pasos.

- (n) `AUX` define $g(t) = f(t) - p_n(t) - [f(x) - p_n(x)]w(t)/w(x)$. ¿Qué pasa si x es un nodo? ¿Quién es responsable de que la demostración sea correcta?

Tu primera entrada en la biblioteca

Abre `..\docs\curso\lab03\mi_grafo.py` y pasa tu demostración de (d) a pasos `Step`: en `deps` van las claves de los pasos que cada paso usa. Agrega un distractor que sea un error que tú podrías cometer. Revísalo con:

```
.venv\Scripts\python ..\docs\curso\lab03\experimentos.py mi_grafo
```

- (o) ¿Qué distractor escribiste y por qué es tentador? ¿Qué flecha de tu grafo lo descarta?

7 Cierre

85-90 min

Entrega (una por pareja, antes de la próxima sesión):

1. La tabla de (c), la demostración de (d) y la tabla de hitos.

2. Las respuestas (e)-(o) en la hoja de entrega. Una respuesta correcta suele caber en dos líneas.
3. Tu archivo `mi_grafo.py`, renombrado `mi_grafo_<apellido1>_<apellido2>.py`.
4. El párrafo final, **con tus palabras y sin IA**: ¿para qué sirve interpolar y para qué no? Usa lo que viste con Colombia y con China.

Tarea opcional

Corre esta orden y después mueve los dos nodos interiores. ¿Puedes bajar Δ ?

```
.venv\Scripts\python -m interprl lebesgue --nodes=-1,-0.5,0,0.5,1
```

Eso es la fase 3 del proyecto: la orden de abajo busca los mejores nodos. Para comprobar el resultado, la búsqueda imprime una orden con `verify`; cámbiale `python` por `.venv\Scripts\python`.

```
.venv\Scripts\python -m interprl search --n 4
```

Hoja de entrega · Laboratorio 3

Nombres: _____

Fecha: _____ Semillas usadas: _____

(a) basales con nodos $-1, 0, 1$, expandidos

(b) basales con nodos $0, 1, 3$, expandidos

(d) $L_0 + \dots + L_n = 1$: cada paso, una afirmación y su razón

(e) Vietnam: tus estimaciones de 2013 y 2015, su error, y por qué funcionó

(f) Colombia 2020: qué cree el polinomio, qué pasó, por qué la fórmula del error no salva

(g) China: error de 2022 y 2023, y cuánto mueve 2023 un error del 1 % en el dato de 2021

(h) qué significa $Q(\gamma = 1) \gg Q(\gamma = 0)$ y su relación con Λ

(i) por qué $|p_n - p_{n-1}|$ es buen estimador, cuándo engaña, cuántas entregas prematuras

(j) dónde fallan los equiespaciados; cuánto cae `so_lo_equi`; por qué rendirse es racional

(k) sin Runge: ¿sigue prefiriendo Chebyshev? ¿propiedad de los nodos o de los problemas?

(l) entregar mal no cuesta: qué aprende y por qué es racional

(m) qué tendría que pasarles a los nodos para que `D_TAYLOR` fuera cierto

(n) qué pasa si x es un nodo; quién es responsable

(o) tu distractor, por qué es tentador y qué flecha lo descarta

Párrafo final, sin IA: ¿para qué sirve interpolar y para qué no?